

Exam Language Trainer - Final Recommendation & Architecture

VERDICT: Stick with Compromise

Based on your project requirements, **compromise is the better choice**.

Why Compromise Wins for Your Project

1. Web Application Focus

-  Works in browser (client-side) AND Node.js (server-side)
-  Small bundle size (~200KB vs wink's ~2.5MB)
-  Faster page loads for international students
-  Can process text client-side (reduces server costs)

2. You Already Have It Working

-  Don't waste time rewriting working code
-  Focus on features that matter (quiz quality, UX)
-  Faster to production

3. Good Enough for Word Extraction

-  Both libraries extract words equally well
-  Neither has built-in "difficulty detection"
-  You'll need custom logic anyway (which I provided)

4. Claude API Does the Heavy Lifting

-  Claude generates the actual quiz
-  Claude can help identify difficult words
-  The NLP library is just preprocessing

Recommended Architecture

FRONTEND (React/Vue)

- File upload (PDF/TXT)
- Display extracted words
- Lecturer selection interface
- Quiz display



BACKEND (Node.js/Express)

1. PDF/TXT Processing

- Extract text from PDF (pdf-parse)
- Clean text



2. Word Extraction (compromise)

- Extract all words
- Filter by difficulty score
- Get POS tags
- Rank by difficulty



3. Lecturer Review

- Display words with difficulty scores
- Allow selection/deselection
- Save selections to database



4. Quiz Generation (Claude API)

- Send selected words to Claude
- Include exam context
- Specify student level (A2/B1/B2/C1)
- Generate quiz JSON



5. Quiz Delivery

- Store quiz in database
- Track student progress



Recommended Tech Stack

Frontend

- **Framework:** React (you mentioned vanilla JS, but React helps with state)
- **UI:** Tailwind CSS or Material-UI
- **File Upload:** react-dropzone
- **PDF Display:** react-pdf (if you want to show PDFs)

Backend

- **Server:** Express.js (Node.js)
- **PDF Processing:** pdf-parse or pdf.js
- **Word Extraction:** compromise ✓
- **Claude Integration:** @anthropic-ai/sdk
- **Database:** Firestore (you mentioned it)

Key Libraries

```
json
{
  "dependencies": {
    "express": "^4.18.2",
    "compromise": "^14.10.0",
    "@anthropic-ai/sdk": "^0.30.0",
    "pdf-parse": "^1.1.1",
    "firebase-admin": "^12.0.0",
    "cors": "^2.8.5",
    "dotenv": "^16.0.3"
  }
}
```

Enhanced Difficulty Detection Strategy

Instead of just relying on compromise, use a **multi-factor approach**:

1. Word Frequency Lists (Most Important!)

Use COCA (Corpus of Contemporary American English) or Google's frequency lists:

- Words in top 1000: Easy (skip)
- Words 1000-5000: Medium
- Words 5000+: Difficult

Download frequency lists:

- <https://www.wordfrequency.info/>
- Google's 10,000 most common: <https://github.com/first20hours/google-10000-english>

2. Academic Word List (AWL)

- 570 word families essential for academic English
- Perfect for your international student use case
- Download: <https://www.wgtn.ac.nz/lals/resources/academicwordlist>

3. Word Characteristics (from compromise)

- Length (longer = harder)
- POS tags (abstract nouns harder than verbs)
- Academic prefixes/suffixes
- Multi-syllable words

4. Let Claude Help! (Best approach)

Send all extracted words to Claude and ask:

```
javascript

const prompt = `
Rate these words by difficulty for B2 level English learners (1-100):
${words.join(', ')}

Return JSON: { "word": "difficulty_score" }
`;
```

Claude knows language difficulty better than any algorithm!

Implementation Priority

Phase 1: MVP (2-3 weeks)

1.  File upload (PDF/TXT)
2.  Basic word extraction with compromise
3.  Simple difficulty scoring (length + frequency list)
4.  Lecturer selection UI
5.  Claude API quiz generation
6.  Basic quiz display

Phase 2: Enhancement (2-3 weeks)

1. Multi-word phrase extraction
2. Better difficulty algorithms (AWL, COCA)
3. Quiz variety (multiple question types)
4. Student progress tracking
5. Quiz analytics

Phase 3: Polish (1-2 weeks)

1. PDF highlighting (show words in context)
2. Word usage examples
3. Difficulty calibration based on student performance
4. Export functionality

Code Structure

```
exam-language-trainer/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── FileUpload.jsx
│   │   │   ├── WordList.jsx
│   │   │   ├── QuizDisplay.jsx
│   │   │   └── Results.jsx
│   │   ├── services/
│   │   │   ├── api.js
│   │   │   └── firebase.js
│   │   └── App.jsx
│   └── package.json
├── backend/
│   ├── routes/
│   │   ├── upload.js
│   │   ├── extract.js
│   │   ├── quiz.js
│   │   └── results.js
│   ├── services/
│   │   ├── pdfProcessor.js
│   │   ├── wordExtractor.js    # compromise here!
│   │   ├── difficultyScorer.js
│   │   └── quizGenerator.js    # Claude API here!
│   ├── utils/
│   │   ├── frequencyLists.js
│   │   └── academicWordList.js
│   ├── server.js
│   └── package.json
├── data/
│   ├── word-frequency-5000.json
│   └── academic-word-list.json
└── README.md
```

When to Consider Wink-NLP

Only switch to wink-nlp if:

- ❌ You move to 100% server-side processing (no browser)
- ❌ You need very precise POS tagging
- ❌ Bundle size doesn't matter
- ❌ You're processing massive documents server-side

For a web app? Stick with compromise.

Cost Considerations

Claude API Costs (Important!)

- Input: ~\$3 per million tokens
- Output: ~\$15 per million tokens

Optimization strategies:

1. Cache common quizzes
2. Batch process multiple words in one API call
3. Use system prompts to reduce token usage
4. Consider Claude Haiku for simple quizzes (cheaper)

Estimated costs per quiz:

- 10 words $\times$ ~500 tokens per quiz = ~5000 tokens
- Cost: ~\$0.02-0.05 per quiz
- 1000 quizzes/month = \$20-50/month

Final Thoughts

Your project is really well-suited for:

1. **Compromise** for word extraction (fast, lightweight, web-friendly)
2. **Claude API** for quiz intelligence (the real value-add)
3. **Frequency lists** for better difficulty detection
4. **Firestore** for scalable data storage

Focus on:

- Great UX for lecturers (easy selection, preview)
- High-quality quizzes from Claude
- Student engagement and tracking

Don't worry about switching NLP libraries. Compromise is perfect for your needs!

Need Help With?

Let me know if you want code examples for:

- PDF text extraction
- Firebase/Firestore integration
- Frontend components
- Claude API prompt engineering
- Difficulty scoring algorithms
- Quiz types and variations

Good luck with your capstone project! 🚀